The Guardian Layer

Problem, trust model, architecture, technology and repository structure

TSYP14 Technical Challenge · City Brain Convergence
Phase 1 · Guardian team working document
Prepared 24 October 2026 · Phase 1 submission due 7 November 2026 · CSTAM pitch 14 November 2026

Contents

#	Section
1	The problem we are solving
2	What we need from the other layers, and what we give back
3	How we can say “we trust this output”: the trust layers
4	Pattern recognition from previous data
5	Trusting actuators: ordered vs. happened
6	The cyber-attack layer
7	Containment
8	Layers to implement and their priority
9	Architecture
10	Technology stack
11	Repository structure for guardian/
12	Roadmap to 14 November
13	Test Book scenarios
14	Open items to settle with partners
15	Known limitations to state honestly

Mission The brief gives us one key question: **“Is this data telling the truth?”** Everything in this document exists to answer it with evidence we can log, explain and show on the timeline screen.

1. The problem we are solving

A smart city only works if three things hold together: **sensing** (Twin), **deciding** (City Brain) and **trusting** (Guardian). Most systems do the first two and assume the data is honest. Because our three layers are built by independent teams, and because data can be falsified (fake flood, replayed readings, emergency-call floods, dead sensors), the City Brain cannot act safely on raw readings. The Guardian sits between what the city reports and what the city does.

What the brief requires of the Guardian

Requirement	What it means for us
Detect	Sensor malfunction, actuator failure, spoofed data, cyber-attacks
Contain	Isolate a compromised sensor, quarantine a device, roll back a falsified reading
Give a trust verdict	Provide the Brain with a verdict on a reading **before** the Brain is allowed to act on it
Stay interoperable	Work with partners we did not build; Phase 2 reshuffles triplets (15 of the 50 Phase 1 points)
Be visible	One scenario = one trigger + one log (R2); a live timeline of reading → verdict → decision → action (R3)

How Guardian is scored in Phase 1 (50 points per team)

Criterion	Pts	How the Guardian earns it
Individual prototype functionality	10	Detection and containment work alone, against our mocks (R1)
Technical implementation & features	5	Six trust layers, actuator verification, threat correlation
Interoperability, connectivity & collaboration	15	Adapters, config-driven partners, Interface Card, input validation, joint run proof
Security & trust mechanisms	10	HMAC, replay protection, signed verdicts, fail-safe behaviour, containment
Documentation & code quality	5	Report part 3 with six subheadings, tests, clean folder
Demonstration & presence	5	Live timeline showing cause and effect

2. What we need from the other layers, and what we give back

Each Guardian function exists because of a specific input or output on the other layers. This table is the reasoning behind our design: we do exactly what the triplet needs from us, and we ask the others for exactly what we need to do it.

Flow	Content	Why the Guardian needs or produces it
Twin → Guardian	Every sensor reading in the common envelope (`sensor_id`, `zone_id`, `timestamp`, `sequence_no`, `value`, HMAC `provenance`)	This is the evidence. Without sequence numbers we cannot detect replays; without the HMAC we cannot prove origin.
Twin → Guardian	Actuator results (`status`, `current_state`, `reverts_at`)	We compare what was ordered with what happened. This is how we detect actuator failure.
Twin → Guardian	Heartbeats, or the absence of readings	Dropout detection (no reading within 2× the expected interval).
Twin → Guardian	Zone metadata (type, features, adjacency)	Needed for cross-checks and semantic plausibility (e.g. a fire in the sea).
Brain → Guardian	A verdict request before acting, or actuator calls carrying `trust_ref` and `justification`	This is the gate: the Brain may only act on a reading the Guardian has judged.
Guardian → Brain	Trust verdict per reading with a score and reasons; security alerts	Brain decisions become safe: ignore untrusted data, corroborate suspect data.
Guardian → Twin	`quarantine_device`, `isolate_sensor`, `clear_quarantine`, rollback to last-known-good	Containment goes through the Twin's actuators, so the city really changes.
Guardian → everyone	Audit events for the timeline screen (R3) and logs (R2)	Judges must see cause and effect.

Verdict format (proposal for the Interface Card)

```
{
  "verdict_id": "verdict-88213",
  "reading_ref": {"sensor_id": "wl-port-03", "sequence_no": 4821},
  "status": "untrusted",
  "trust_score": 0.08,
  "reasons": ["rate_of_change", "no_rainfall_corroboration"],
  "action_taken": "isolate_sensor",
  "ts": "2026-11-02T14:03:23Z"
}
```

Status	Meaning	Rule for the Brain
`trusted`	Passed all hard gates and scored high with corroboration	Act
`suspect`	Something is off, or there is not enough independent evidence	Require corroboration before acting
`untrusted`	Failed a hard gate or scored low	Ignore
`unknown`	Stale, missing data, or the verification source is itself untrusted	Treat as not trusted (fail-safe)

3. How we can say “we trust this output”

Trust is never absolute, so we define it precisely: **a reading is trusted when it passes independent layers of evidence, and that evidence is logged.** Cheap, definitive checks run first; expensive statistical checks run last. A hard failure at layers 1 or 2 makes a reading `untrusted` immediately. Layers 3 to 6 accumulate into a trust score.

#	Layer	Question it asks	Check	Catches
1	**Authenticity**	Is it genuine and well-formed?	Schema validation, `contract_version`, HMAC verification	Tampering, injection, malformed data
2	**Freshness**	Is it new?	`sequence_no` strictly increasing, timestamp skew	Replay attacks
3a	**Range & unit**	Is the value physically possible?	Range, units, internal consistency (AQI 0–500, `load_kw ≤ capacity_kw`)	Faulty values, unit mix-ups between teams
3b	**Semantic plausibility**	Is this event possible here?	City model: fire in open sea, flood where no water body exists	Impossible context, crude spoofing
4	**Dynamics**	Does the change over time make sense?	Rate of change, stuck-at / flatline, EWMA / z-score, per-hour baseline	Fake spikes, frozen sensors, drift
5	**Corroboration**	Does the rest of the city agree?	Cross-check independent sources and neighbouring zones	Coordinated or subtle spoofing
6	**Liveness / rate**	Is the device alive and at a normal pace?	Heartbeat, per-zone arrival rate, duplicate IDs	Dead devices, call floods (>5 calls/zone/min)

Layer 1: Authenticity

Schema validation checks that all envelope fields exist with correct types. A `contract_version` mismatch is reported with a clear error rather than silently misread. The HMAC is recomputed over the payload and compared in constant time. **Limit:** HMAC proves the message came from a key holder and was not altered; it does not prove the value is true. A compromised sensor signs lies perfectly, which is why layers 3 to 6 exist.

Layer 2: Freshness

`sequence_no` must increase strictly per sensor, and the timestamp must be close to our clock. A repeated number (4821, 4821) is a replay. An attacker replaying a legitimate old message still passes the HMAC, so only the sequence number exposes it. **Careful:** a Twin restart may reset counters, so we need an agreed “sequence reset” rule to avoid false alarms.

Layer 3a: Range and unit plausibility

Check	Example violation
Range	`aqi: 900` (scale is 0 to 500)
Internal consistency	`load_kw: 620` with `capacity_kw: 500`
Units	`level_m: 180` (that is centimetres, not metres)
Sign and type	`vehicle_count: -3`, `congestion_score: 1.7`

Limit: an attacker who stays inside the valid range passes this layer. A fake 180 cm flood passes if the sensor's maximum range is 300 cm.

Layer 3b: Semantic plausibility (world-model check)

Plausibility as a value check does not ask whether the event is possible **here**. A fire in the sea has valid numbers but an impossible context. The check compares a reading or incident with static knowledge of the city, kept in `config/city_model.yaml` so nothing is hard-coded.

Check	Example violation
Location sanity	Fire or smoke call from an open-sea zone, or a park with no buildings
Sensor-environment fit	Water-level sensor in a zone with no water body; 80 vehicles in a pedestrian-only zone
Time and context	Rush-hour congestion at 3 a.m. with no event
Physical coupling	Flood of 1.8 m in a zone 30 m above the river
Capability	Grid-disconnect order on a zone with no switchable grid; fire unit sent where there is no road access

```
zone-07:                      zone-12:
  type: port                    type: open_sea
  has_water_body: true          has_buildings: false
  max_water_level_m: 3.0        fire_possible: false
  has_buildings: true
  road_access: true
```

> **Design decision** We keep the feasibility knowledge **inside the Guardian**, not by asking the City Brain. The Guardian's value is being an independent second opinion: if it asked the layer that consumes the data to judge it, a fooled or compromised Brain would fool the check too. It would also couple us to a partner and weaken R1. The zone model is a Twin-side decision, so we ask the Twin team for a zone-metadata endpoint, and otherwise keep a minimal copy in `config/` as a documented assumption. A semantic failure is `suspect` unless it is physically impossible (the sea fire), because an incomplete map would cause false alarms.

Layer 4: Dynamics

Looks at the history of a single sensor: rate of change (water cannot rise 0.2 m in 5 s with no cause), flatline detection (50 readings of exactly `0.420000` means a frozen or replayed sensor), and an EWMA / z-score against a running baseline. A dynamics failure alone gives `suspect`, because a real event such as a burst pipe also looks abrupt; layer 5 settles it.

Layer 5: Corroboration (the strongest layer)

A lone attacker usually controls one sensor, not all of them, so we compare a reading with independent sources that should correlate.

Reading	Cross-checked against
Water level rising	Rainfall sensor, upstream sensors, flow rate
AQI spike	Smoke / fire flag, camera, neighbouring zone AQI
Congestion score high	Average speed, vehicle count, transit delay
Power-load jump	Grid frequency / voltage, neighbouring zone loads
Emergency fire call	Smoke detection, AQI, a second caller

```
wl-port-03 says 1.80 m (rising fast)
  rainfall sensor:    0 mm/h     ✗ no rain
  flow_rate:          unchanged  ✗ no extra flow
  neighbouring zone:  normal     ✗ no spread
  → 3 independent sources disagree → untrusted, reason: no_corroboration
```

A real flood passes: rain is high, flow rises, upstream sensors rose minutes earlier. **Limit:** corroboration needs at least two independent, currently trusted sources; an isolated sensor can only be `suspect`.

Layer 6: Liveness and rate

Heartbeat: no reading within 2× the expected interval means `unknown`/stale. Arrival rate: more than 5 emergency calls per zone per minute is a flood. Duplicate `call_id` means a duplicate or replay. A flood does not prove every call is fake (a real disaster also produces many calls), so we throttle and mark calls for corroboration instead of dropping them.

How the layers combine into one verdict



Exact weights are a design choice we justify in the report and tune in testing: a plausibility failure is a strong penalty, a dynamics failure a medium penalty, failed corroboration a strong penalty and passed corroboration a strong bonus.

Why layers instead of one smart detector

- **Different attackers beat different layers.** A replay beats the HMAC; a smart spoofer beats plausibility; a compromised Twin beats authenticity. Stacking forces an attacker to beat all of them.
- **Explainability.** The verdict lists reasons, so judges and the Brain see why. That is far easier to show on a timeline than a black-box score.
- **Cost ordering.** Cheap checks reject bad data early, so expensive cross-checks run only on survivors.

4. Pattern recognition from previous data

Layer 4 uses simple statistics. Pattern recognition is its next level: instead of asking “is this value an outlier?”, it asks “**does this sensor behave like itself?**”

Pattern	Example	What it catches
Daily / weekly seasonality	Traffic peaks at 8:00 and 17:30; power load drops at night	A “rush-hour jam” at 3 a.m.; flat load during the day
Sensor fingerprint	Each sensor has its own noise level, update jitter and typical range	A spoofer sending “too clean” data; a replaced device
Correlation structure	Rain rises, then water level rises about 10 minutes later	A level that rises with no lag, or before the rain
Sequence shape	Real floods ramp up over minutes; fake spikes are step functions	Crude injected attacks

How to build it, in order of effort

- **Baseline profile per sensor and hour of day** (mean and standard deviation per time bucket). Cheap, explainable, probably 80% of the value.
- **Isolation Forest or robust z-score** on features such as level, rate of change, noise and cross-sensor deltas.
- **Optional:** a small autoencoder or LSTM. We skip this unless the first two are done: it needs much data and is hard to explain to judges.

Rules for the ML part

- It can **lower trust or raise** `suspect`**, never make a reading** `trusted` **on its own.**
- It always returns a reason, for example “value 4.2σ above the Tuesday 14:00 baseline”.
- **Training data:** ask the Twin team early for a normal-day recording or an endpoint that generates N hours of clean data; otherwise learn online with a learning mode (vulnerable if an attack starts at the very beginning).
- **Phase 2:** a new Twin means a new baseline, so models train from config and data and are never hard-coded to one Twin.
- **Poisoning defence:** an attacker who drifts a value slowly teaches the baseline that the fake is normal. We freeze the baseline from verified-clean data and only update it with readings that were `trusted`.

5. Trusting actuators: ordered vs. happened

An actuator call has two parts: the **order** (what the Brain asked for) and the **outcome** (what actually changed). A healthy actuator makes them match; an actuator failure is any case where they do not. The Twin's reply (`status: ok`) only says the command was **accepted**. A stuck valve, a hacked controller or a buggy Twin can answer “ok” and do nothing, so we verify the effect ourselves with sensors that are independent of the actuator.

The four checks

#	Check	Detail
1	**Authorisation and provenance**	The call has a valid `trust_ref` pointing to a verdict that was `trusted` and not expired. A call based on an untrusted or quarantined device is blocked.
2	**Policy and safety invariants**	No grid disconnect on a hospital zone, no duplicate dispatch for the same `incident_id`, command rate limits.
3	**Effect verification**	After a command (e.g. `closed`), we expect the physical effect within a timeout. If it does not happen: `actuator_failure`.
4	**Audit**	Every call is logged with its justification.

Worked example: closing a valve

```
t=0s    ORDER      set_valve_state valve-port-12 → closed
t=0.2s  REPLY      {status: "ok", current_state: "closed"}
                   ↳ expectation stored: "flow_rate_m3s should fall below 10%
                     of its pre-command value within 30 s"
t=30s   OBSERVED   flow_rate_m3s is still 1.2 (was 1.2)
                   ↳ VERDICT: actuator_failure
                     reason: "valve reported closed, flow unchanged"
```

The Guardian then flags the valve as failed and tells the Brain not to rely on it, so the Brain can pick a backup (another valve, a unit dispatch).

Expected effect per actuator

Actuator	Order	Expected effect, observed by a different source
Water valve	`closed`	Flow rate or level trend changes downstream
Grid switching	`disconnected zone-07`	`load_kw` on that zone's meter drops to about 0
Traffic signal	`green for 60 s`	Vehicle count and speed on the matching camera change; phase reverts at `reverts_at`
Dispatch	`dispatch_unit`	A unit ID and ETA are returned; unit shows for that `incident_id`; no duplicate
Quarantine	`quarantine_device`	No more readings from the device are used

The verification sensor must be independent of what we check. Verifying a valve with its own status field would let a lying actuator confirm itself.

Failure types this catches

Mismatch	Example	Likely cause
No effect	Valve “closed”, flow unchanged	Stuck actuator, or a Twin that lies about success
Wrong effect	Asked to cut zone-07, zone-08 went dark	Wrong target, bug or malicious redirect
Partial effect	Target 0%, flow fell only 40%	Degraded actuator
Late effect	Effect after 3 minutes instead of 30 s	Slow or struggling device
Reply contradicts order	Asked `closed`, reply says open	Rejection ignored or faulty reply
Unauthorised effect	State changed and no call was ever issued	Rogue command or an attacker acting directly
Reverted early	Signal should stay green 60 s but reverts at 10 s	Controller fault or interference

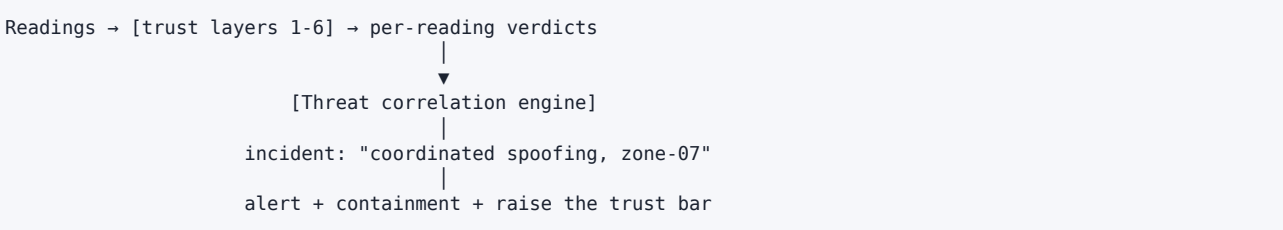
Expectations carry a deadline and a tolerance in `config/policies.yaml`, which avoids false alarms from noise and delay:

```
set_valve_state:
  closed:
    verify_with: [flow_rate_m3s]
    expect: "drop_pct >= 80"
    within_s: 30
    on_fail: actuator_failure
```

Subtlety The verifier can only conclude “actuator failure” if the verification sensor is itself **trusted**. If the flow sensor is `untrusted`, “no effect observed” might mean a bad sensor rather than a bad valve, so we return `unknown` instead of accusing the actuator.

6. The cyber-attack layer

The trust layers judge **individual readings**. An attack is often about behaviour across many readings and devices, which no single-reading check sees. The brief lists cyber-attacks separately from spoofed data, and Security & Trust is 10 of the 50 points. The threat layer is a **second stage above the trust layers**: it reads verdicts and traffic and decides “an attack is happening”.



Attack taxonomy

Attack	Signals	Response
Replay	Repeated `sequence_no`, valid HMAC on old data	Block, flag the source
Spoofing / false data injection	Failed corroboration, impossible dynamics	Isolate sensor, roll back to last-known-good
Coordinated multi-sensor attack	Several sensors in one zone fail checks within a short window	Zone alert, “high-distrust mode” for the zone
DoS / flood	Message-rate spike, emergency call flood	Rate-limit, throttle the source
Tampering / man-in-the-middle	HMAC failures, schema violations	Reject, alert on possible key compromise
Command injection / actuator abuse	Calls with no `trust_ref`, bursts of calls, unusual actions at odd times	Block, quarantine the caller
Compromised Twin	Everything signed correctly but corroboration collapses across the city	Escalate: tell the Brain to fall back to safe mode
Slow drift / poisoning	Gradual deviation from baseline over a long window	Alert, freeze the baseline
Attack during a real incident	Real flood plus a fake alarm elsewhere	Keep acting on trusted data, ignore untrusted

How it works

- **Event correlation:** group verdicts by zone, device, source and time window. “3 failures in 20 s in zone-07” is an incident; one failure is noise.
- **Attack-pattern catalogue:** signatures kept in `config/attack_patterns.yaml`.
- **Threat level per zone:** `normal → elevated → under_attack`. At higher levels thresholds tighten (e.g. `trusted` needs two corroborating sources instead of one).
- **Security alert** emitted to the Brain and the timeline, with attack type, evidence and suggested response.

Two design points

- **Response must be proportionate.** Over-aggressive quarantine lets an attacker use our defence against us (spoof a fake attack to get healthy sensors isolated). We use hysteresis, a maximum number of isolations per zone, and a release path (`clear_quarantine`).
- **The Guardian can be attacked too.** Its inputs, config, keys and verdict channel are protected with signed verdicts and an authenticated API. This is part of the “security measures” section of the report.

7. Containment

Action	What it does	Guardrail
`isolate_sensor`	Stops the Brain and other layers from using a sensor's readings	Cap per zone; log the evidence
`quarantine_device`	Blocks a device (sensor, actuator or caller) from acting or being used	Reversible; reason required
Rollback	Replaces a falsified reading with the last-known-good value	Marked clearly as rolled back in the audit log
`clear_quarantine`	Releases a device after it behaves normally again	Hysteresis to avoid flapping

Containment calls go to the Twin's actuators (`requesting_layer: guardian`; `trust_ref` may be null because the Guardian can self-authorise containment). Every action writes a `justification` to the audit log.

8. Layers to implement and their priority

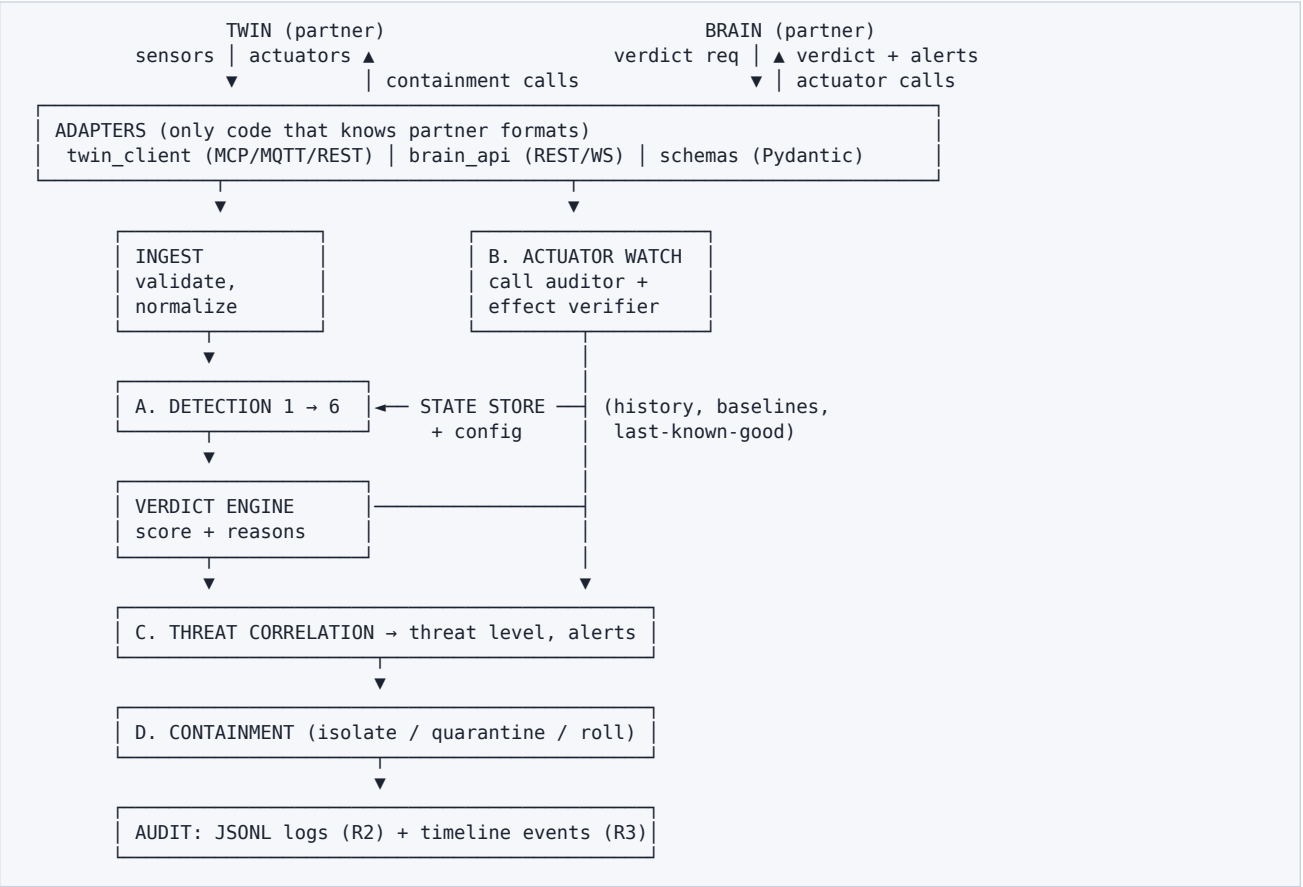
Block	Brief requirement	Contains
A. Data trust	“Is this sensor telling the truth?”	Seven detection layers (1, 2, 3a, 3b, 4, 5, 6)
B. Actuator trust	Actuator failure	Call auditor and effect verifier
C. Threat detection	Cyber-attacks	Correlation engine, attack catalogue, threat levels
D. Containment	Isolate, quarantine, roll back	Containment manager
Cross-cutting	Trust verdict to the Brain; visible cause and effect (R3)	Verdict engine, audit and timeline emitter
Cross-cutting	Interoperability (15 pts)	Adapters, config, mocks

Priority per layer

Layer	Priority
1 Authenticity (schema, `contract_version`, HMAC)	**Must**
2 Freshness (`sequence_no`, timestamp skew)	**Must**
3a Range, unit, consistency	**Must**
3b Semantic plausibility (`city_model.yaml`)	**Should**
4 Dynamics (rate of change, flatline, EWMA / z-score)	**Must** (per-hour baseline: should)
5 Corroboration (cross-sensor, zone adjacency)	**Must**
6 Liveness and rate	**Must**
B Actuator watch (auditor + effect verifier)	**Must**
C Threat correlation (rule-based: replay, flood, coordinated spoofing, command abuse)	**Must**
D Containment (isolate, quarantine, rollback, clear)	**Must**
Optional ML anomaly detector (Isolation Forest), LLM sanity advisor	**Nice to have**

Principle A complex ML model that fails live on stage costs more than it earns. A simple, explainable detector that visibly catches the attack on the timeline screen is what the jury scores. The optional ML and LLM modules can only lower trust, never raise a reading to `trusted`, and are never in the critical path.

9. Architecture



Two flows

- **Reading flow (continuous):** Twin → ingest → layers 1 to 6 → verdict → state store → Brain, with threat correlation consuming verdicts.
- **Decision flow (on demand):** Brain asks for a verdict or sends an actuator call → auditor checks `trust_ref` and policy → forward or block → effect verifier watches the outcome.

Pipeline design rules

- **Async and event-driven**, so readings never block each other.
- **Verdict latency budget** under 500 ms for cached or recent data, to agree with the Brain team.
- **Fail-safe:** if the Guardian is unreachable or a verdict is missing, the Brain treats data as `unknown` (not trusted). Documented in `INTERFACE.md`.
- **Stateless where possible:** state sits in the store, so the process can restart and recover.

10. Technology stack

Concern	Choice	Why
Language	**Python 3.11+** with `asyncio`	Best fit for detection logic and ML; fast to write before 7 Nov
Twin-side transport	**MCP Python SDK** client; **MQTT** (`aiomqtt`) if the Twin streams	Brief says the Twin exposes “MCP-style” tools; MQTT suits continuous streams
Brain-facing API	**FastAPI** (REST) with OpenAPI docs	Standard, self-documenting, auto-validated; good for the Interface Card
Live push	**WebSocket** (FastAPI)	Verdict and alert stream to Brain and timeline
Validation	**Pydantic v2**	Strict schemas, clear error messages (a jury criterion)
Crypto	`hmac` / `hashlib` (SHA-256), `secrets`	Standard library, constant-time compare, no custom crypto
State store	In-memory ring buffers (`deque`) + **SQLite**	No extra infrastructure; runs on stage with no install
Stats / patterns	`numpy`, `pandas`	Baselines, EWMA, z-score; light and explainable
Optional ML	`scikit-learn` (Isolation Forest)	Small, trains fast, no GPU
Optional LLM advisor	Local or API model behind a switch	Off the critical path; can only lower trust
Config	**YAML** (`PyYAML`) + `.env` for secrets	No hard-coded partner addresses (interoperability)
Logging	`structlog` / JSON logging to **JSONL**	Machine-readable logs for R2; feeds the timeline
Timeline UI	Shared HTML/JS page on WebSocket	R3 is a triplet-level screen; we only emit events
Tests	`pytest`, `pytest-asyncio`, `hypothesis`	Unit tests per detector; fuzzing for malformed inputs
Packaging	**Docker** + shared `docker-compose.yml`	One-command start; nothing installed on stage
Quality	`ruff`, `mypy`, `pre-commit`	Scores on code quality

Why this stack: it is built from standard protocols and formats (REST, WebSocket, MCP, MQTT, JSON, YAML), exactly what the interoperability criteria reward. It needs no heavy infrastructure, so it runs offline with a local fallback on stage. Nothing in it is tied to one Twin or Brain, so Phase 2 partners need only an adapter and a config change.

What we avoid: Kafka, Kubernetes, deep-learning frameworks and external databases. They add demo risk and setup time without scoring more points.

11. Repository structure for guardian/

The spec fixes what each team folder contains: `README.md`, `INTERFACE.md`, `src/`, `mocks/` and `tests/`. We keep those five as the skeleton and add `config/` so partner addresses and thresholds are never hard-coded.

```
guardian/
├── README.md              # how to run the Guardian ALONE (with mocks)
├── INTERFACE.md           # Interface Card (R1): messages, examples, auth, how to repoint partners
├── Dockerfile             # so the root docker-compose can start us
├── .env.example           # secret placeholders (HMAC key, tokens). Never commit real ones
├── config/                # everything partner- or tuning-specific; never in code
│   ├── guardian.yaml         # partner URLs/ports, transport, timeouts
│   ├── sensor_registry.yaml  # per type: fields, units, expected interval, applicable checks
│   ├── thresholds.yaml       # starting thresholds from the data dictionary, tunable
│   ├── crosschecks.yaml      # which sensors corroborate which, zone adjacency
│   ├── city_model.yaml       # zone types and features (semantic plausibility)
│   ├── policies.yaml         # containment rules, actuator expectations, safety invariants
│   ├── attack_patterns.yaml  # attack catalogue signatures
│   └── baselines/            # learned profiles, versioned
├── src/
│   ├── main.py
│   ├── adapters/             # ONLY code that knows partner formats
│   │   ├── twin_client/      #   readings in; quarantine / isolate tool calls out
│   │   ├── brain_api/        #   verdict endpoint + actuator-call intake
│   │   └── schemas/          #   envelope and verdict models, validation, clear errors
│   ├── ingest/               # validator, normalizer, state_store (rolling history)
│   ├── detectors/            # the trust layers
│   │   ├── authenticity.py   freshness.py   plausibility.py
│   │   ├── semantic_plausibility.py   dynamics.py
│   │   ├── crosscheck.py     liveness_rate.py
│   │   └── patterns/         # baselines, isolation forest, seasonality
│   ├── trust/                # scorer.py, verdict_engine.py, signer.py
│   ├── actuator_watch/       # call_auditor.py, effect_verifier.py
│   ├── threat/               # correlator.py, attack_catalogue.py, threat_level.py, alerts.py
│   ├── containment/          # isolate.py, quarantine.py, rollback.py
│   └── audit/                # logger.py (→ root logs/), timeline_emitter.py (R3)
├── mocks/
│   ├── twin_mock/            # normal data + attack generators (spike, replay, stuck, flood)
│   └── brain_mock/           # requests verdicts, issues good and bad actuator calls
└── tests/
    ├── unit/                 # each detector, scorer, containment
    ├── scenarios/            # Guardian-side scenario scripts against the mocks
    └── fixtures/             # sample readings and verdicts (also used in INTERFACE.md)
```

How it maps to the triplet repository

Item	Where it lives / who owns it
Isolation (R1)	`mocks/` lets the Guardian run alone; `adapters/` keeps all partner-specific code in one place. This scores on the interoperability criteria and protects us in Phase 2.
Report mapping	Detectors, trust and containment are separate folders, each explainable as one report section and testable on its own. Report Part 3 uses the six subheadings: role and goals, architecture, tech stack, interfaces in/out, security measures, known limitations.
Ownership	We commit only inside `guardian/` (enforced by CODEOWNERS).
Shared root files	`scenarios/`, `logs/`, `docker-compose.yml`, `INTEGRATION.md`, `TESTBOOK.md` are agreed with the other two teams. We write scenario scripts in `guardian/tests/scenarios/` and share the trigger commands for the root `scenarios/` folder.
Logs (R2)	Our audit logger writes to the root `logs/` with one agreed pattern, e.g. `logs/t5-*.log`.

12. Roadmap to 14 November

Date check The Phase 1 specification sheet says submission on **7 November**, while the main brief lists a Phase 1 deadline of 12 November. We plan for the earlier date and confirm with the organisers (yp@ieee.tn, cs@ieee.tn or cn@ieee.tn).

Dates	Milestone	Deliverable
23–25 Oct	**Freeze the contract with both partners:** envelope, zone model, HMAC key exchange, verdict format, transport, latency budget. Write the first `INTERFACE.md`; build the Twin and Brain mocks.	Interface Card v1, mocks, skeleton running alone (R1)
26–29 Oct	Core pipeline: ingest and validation, layers 1, 2, 3a, 6, verdict API, state store, audit log.	First end-to-end verdict on mock data
30 Oct–1 Nov	Layers 4 and 5, actuator auditor and effect verifier, containment, rule-based threat correlation.	Fake-flood scenario works with mocks
31 Oct	Random triplet matching (Round 1) per the main brief.	Partners confirmed
2–4 Nov	**Integration with real partners.** Run all Test Book scenarios, feed the timeline (R3), generate run logs (R2), tune false alarms.	Joint run logs and evidence for R4
5–7 Nov	Semantic plausibility, baseline, tests, docs, report part 3, pitch slides, freeze, submit.	Submission
9–13 Nov	Jury review period. Bug fixes only.	Stable build
13–14 Nov	Rehearse the demo with a local fallback and our own hotspot. CSTAM: 2-minute pitch + 8-minute live demo.	Ready for CSTAM
Only if stable	Isolation Forest and optional LLM advisor.	Nice to have

To confirm Your triplet is assigned on 24 September and the main brief also lists a Round 1 matching on 31 October. Confirm with the organisers which applies, because it changes who our real partners are in the 2–4 November integration window.

13. Test Book scenarios

The Test Book must cover all three layers and every interaction between them (Twin → Brain, Twin → Guardian, Brain → Guardian, all three). Each scenario uses the same fields: ID, name, layers involved, description, trigger, expected Twin / Guardian / Brain behaviour, and how to verify.

ID	Scenario	Expected Guardian behaviour	Interaction
T1	**Fake flood at the port** (level 180 cm vs real 40 cm)	Marks the reading untrusted within 10 s, isolates the sensor; Brain does not open the valve	All three
T2	**Stuck or dead sensor**	Flatline / heartbeat check flags `suspect` or `unknown`	Twin → Guardian
T3	**Replayed readings**	Repeated `sequence_no` detected despite valid HMAC	Twin → Guardian
T4	**Emergency-call flood**	Arrival-rate limit and duplicate detection; calls throttled and marked for corroboration	Twin → Guardian
T5	**Valve ordered closed but no effect**	Effect verifier reports `actuator_failure`; Brain is told to use a backup	Brain → Guardian
T6	**Brain tries to use a quarantined device**	Call auditor blocks the call	Brain → Guardian
T7	**Twin goes silent**	Liveness check marks data `unknown`; Brain falls back safely	Twin → Guardian
T8	**Fire reported in the open-sea zone**	Semantic check marks it implausible; no fire unit dispatched	Twin → Brain → Guardian
T9	**Coordinated spoofing in one zone**	Threat correlator raises zone to `under_attack` and tightens thresholds	Twin → Guardian
T10	**Command injection with no** `trust_ref`	Blocked, caller quarantined	Brain → Guardian
T11	**Real flood plus attack at the same time**	Real incident handled using trusted data; fake data ignored	All three

Scenarios not run live are checked through logs, so every scenario needs a trigger (one command or button) and a stored log file (R2), and its cause and effect must be visible on the timeline screen (R3).

14. Open items to settle with partners

#	Item	Our recommendation
1	Transport and format for readings and verdicts (the data dictionary is only a proposal)	Freeze `contract_version` after confirming field names and units with Twin and Brain
2	How the HMAC key is shared and rotated	Shared secret via `.env`, documented rotation, constant-time compare
3	Does the Brain **ask** before acting, or consume a verdict stream?	Synchronous gate: the brief says the Brain needs a verdict “before it is allowed to act”
4	Location format for emergency calls	`zone_id` alone, or `zone_id` plus approximate coordinates, depending on Twin precision
5	Zone granularity and adjacency map for cross-checks	Ask the Twin team for a zone-metadata endpoint
6	Sequence-number reset rule after a Twin restart	Documented “sequence reset” event to avoid false replay alarms
7	Normal-day recording for baselines	Request it early; needed for pattern recognition
8	Which extended sensors and actuators are in scope for Phase 1	Required five first; extended set only if time allows
9	Who builds the shared timeline screen (R3)	Agree one owner; each layer only emits events

15. Known limitations to state honestly

Report part 3 requires “known limitations” and “handling of false alarms”. Stating these openly strengthens the report.

- **Compromised Twin.** If the Twin itself is compromised, HMAC alone does not help; corroboration across independent sources is our defence, and it is weaker if many sensors are spoofed at once.
- **Corroboration needs two independent, trusted sources.** An isolated sensor can only be `suspect`, not `trusted`.
- **Threshold trade-off.** Thresholds trade false alarms against missed attacks. We log every decision and tune during integration.
- **HMAC proves origin, not truth.** A compromised sensor signs lies; that is why layers 3 to 6 exist.
- **Real events can look like attacks.** A burst pipe or a real disaster produces abrupt changes and many calls. We throttle and corroborate rather than drop.
- **Incomplete city model.** A wrong zone map causes false semantic alarms, so those failures are `suspect` unless physically impossible.
- **ML and LLM components** can drift, be poisoned or hallucinate. They only lower trust and are never in the critical path; free text from a sensor never reaches an LLM prompt unfiltered.
- **Defence can be abused.** An attacker may try to trigger our quarantine. Hysteresis, per-zone isolation caps and a release path limit this.
- **Guardian outage.** If the Guardian is down, the Brain must treat data as `unknown`; this fail-safe has to be agreed and documented in `INTERFACE.md`.

Next Generate the `guardian/` scaffold (README, `INTERFACE.md` with message examples, YAML configs, module stubs with docstrings and a runnable Twin / Brain mock pair) so the skeleton starts with one command.